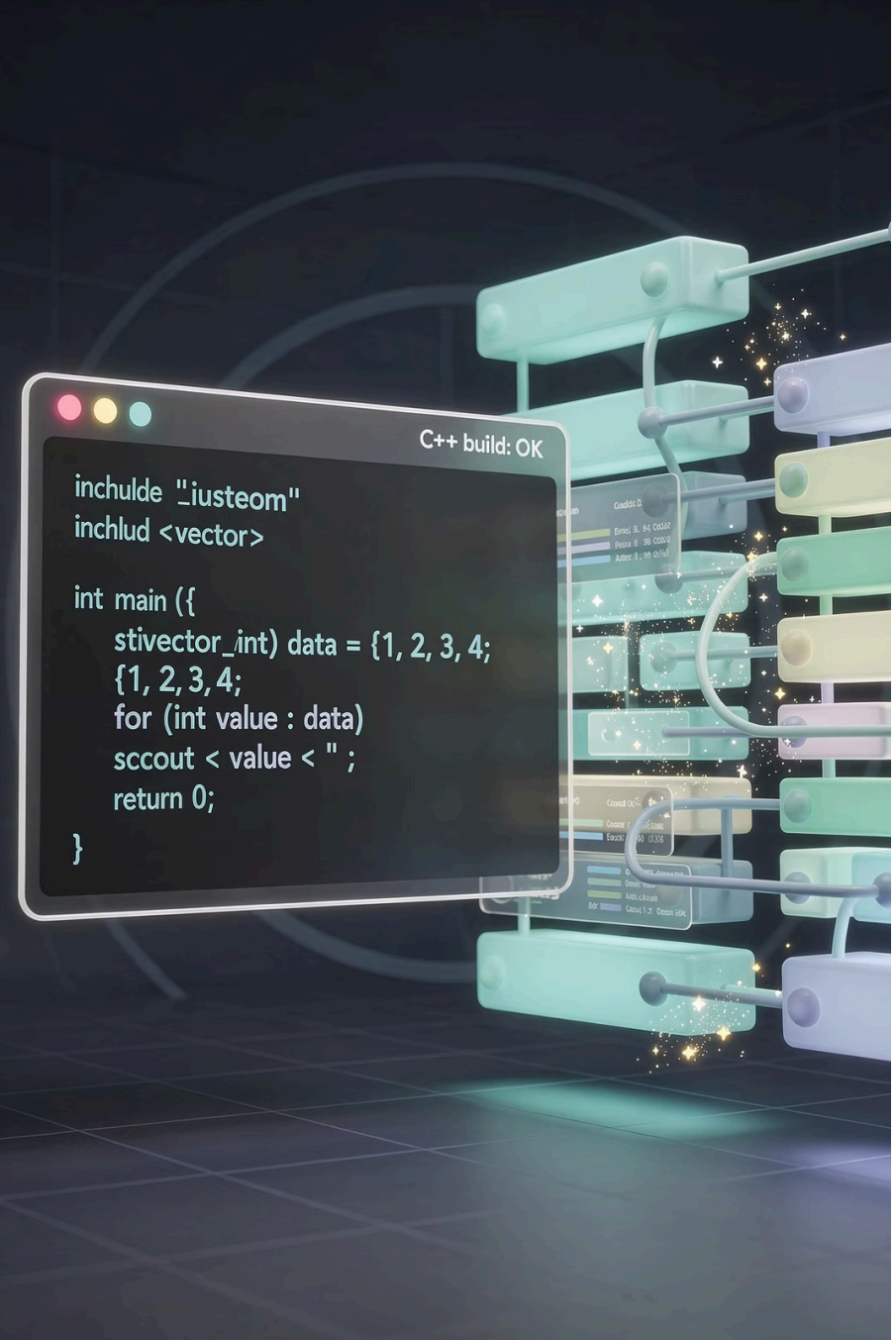


Mastering Resource Management in C++

Copy & Move Constructors, Deep vs. Shallow Copy, and the Rule of Five

C++ FUNDAMENTALS

RESOURCE MANAGEMENT

SMART Learning Objectives

By the end of this presentation, you will be able to:

01

Define Copy & Move Semantics

Clearly distinguish between copy and move operations and their role in C++ resource management.

02

Implement Safe Constructors

Write robust copy and move constructors to manage resources effectively and prevent common errors.

03

Apply the Rule of Five

Understand and correctly implement the Rule of Five (or Zero) for designing classes that own resources.

04

Identify & Mitigate Risks

Recognize potential memory safety issues related to resource handling and apply strategies to resolve them.

Why Proper Copy & Move Semantics Matter

In C++, how objects are copied and transferred is not just a theoretical concern — it directly impacts correctness, performance, and safety. Without careful attention to these mechanics, programs can suffer from memory leaks, dangling pointers, and undefined behavior that is notoriously difficult to debug.

🛡️ Memory Safety

Prevents double-deletes, dangling pointers, and undefined behavior in complex class hierarchies.

⚡ Performance

Move semantics enable efficient transfer of ownership, eliminating costly unnecessary deep copies.

🏗️ Correctness

Proper semantics ensure that objects behave predictably when passed, returned, or assigned.

The Purpose of Copy Constructors

What is a Copy Constructor?

A copy constructor creates a new object as an exact copy of an existing one. It is invoked when an object is initialized from another object of the same type — whether passed by value, returned from a function, or explicitly copied.

Signature: `MyClass(const MyClass& other);`

Default vs. Custom Behavior

- **Default behavior**: The compiler generates a memberwise (shallow) copy, duplicating each member's value including raw pointer addresses.
- **Custom behavior**: You override the copy constructor to perform a deep copy, allocating new memory and copying the pointed-to data — not just the pointer itself.
- **Rule**: If your class owns a raw resource, always define a custom copy constructor.

The Purpose of Move Constructors

Introduced in C++11, move constructors allow the **transfer of resources** from a temporary or expiring object (an rvalue) to a new object — without duplicating the underlying data. This is a cornerstone of modern, high-performance C++.

1

Source Object

A temporary or expiring object holds heap-allocated resources (e.g., a buffer, file handle).

2

Move Constructor Invoked

`MyClass(MyClass&& other) noexcept` is called. Resources are "stolen" from the source.

3

Destination Object

Takes full ownership of the resources. Source is left in a valid but empty (null) state.

Move constructors should be marked `noexcept` to enable STL optimizations — containers like `std::vector` will prefer move over copy only when the move constructor is guaranteed not to throw.

Behavior of Copy Constructors

Shallow Copy (Default)

The compiler-generated copy constructor performs a bitwise copy of all members. For pointer members, this means both the original and the copy point to the **same memory address**. This leads to double-free errors when both objects are destroyed.

```
// Dangerous default copy
MyClass a;
MyClass b = a; // b.ptr == a.ptr!
// Destructor called twice → crash
```

Deep Copy (Custom)

A custom copy constructor allocates **new memory** and copies the contents from the source object. Each object owns its own independent resource, eliminating shared ownership risks.

```
MyClass(const MyClass& other)
 : size(other.size),
 data(new int[other.size]) {
 std::copy(other.data,
 other.data + size, data);
}
```

Behavior of Move Constructors

Move constructors are designed to be fast — they transfer ownership in O(1) time regardless of the size of the resource. The key steps are: steal the pointer, nullify the source, and leave everything in a consistent state.

Step 1: Transfer Resource Pointers

Assign the source's internal pointer (or handle) to the new object: `this->data = other.data;`

Step 2: Nullify Source

Set the source pointer to `nullptr`: `other.data = nullptr;` This prevents the source destructor from freeing the now-transferred memory.

Step 3: Source Remains Valid

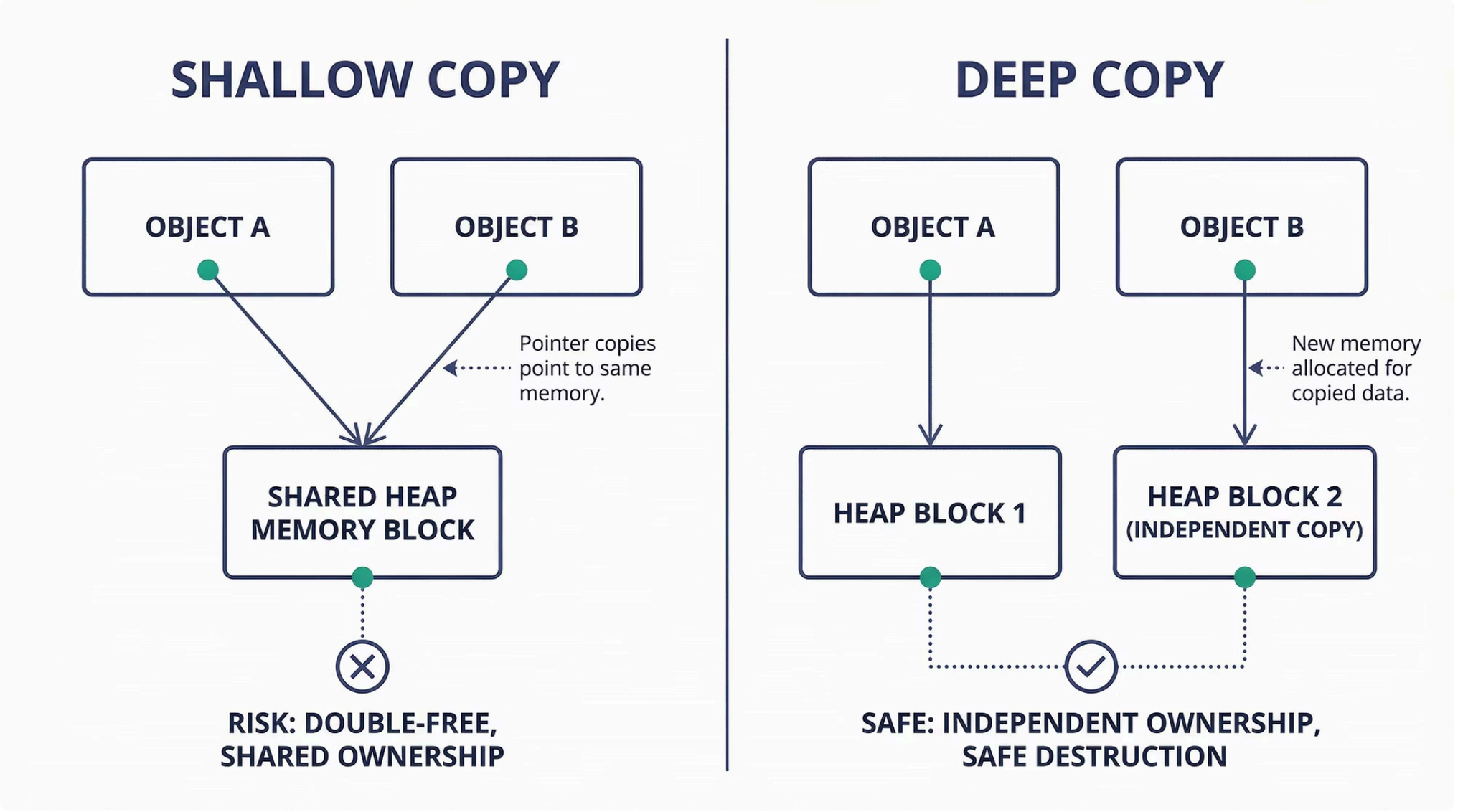
The source object must remain in a **destructible state**. Its destructor will run, but because its pointer is null, no double-free occurs.

Shallow Copy vs. Deep Copy

Choosing between shallow and deep copy is fundamentally about **resource ownership semantics**. Misidentifying which approach to use is one of the most common sources of memory bugs in C++.

Aspect	Shallow Copy	Deep Copy
What is copied	Pointer address only	Pointed-to data (new allocation)
Memory	Shared between objects	Independent per object
Risk	Double-free, use-after-free	Higher memory & time cost
Use when	Shared, immutable, or externally managed resources	Unique ownership; mutable resources
C++ idiom	`std::shared_ptr`	Custom copy ctor or `std::unique_ptr`

Shallow vs. Deep Copy: Visual Overview



In a **shallow copy**, two objects share the same heap memory — when either is destroyed, the other is left with a dangling pointer. In a **deep copy**, each object owns a separate allocation, so destruction of one has no effect on the other. Understanding this distinction is the foundation of safe C++ resource management.

When to Use Shallow Copy

Shallow copy is appropriate when objects do not own their resources exclusively, or when the resource is managed by an external authority. The key question is: *"If I destroy one copy, will the other become invalid?"*

Immutable Objects

If the pointed-to data is never modified, sharing a pointer between copies is safe. No writes, no race conditions, no corruption.

Externally Managed Resources

When another entity (e.g., a memory pool or reference-counted system) controls resource lifetime, shallow copy is safe and efficient.

Performance-Critical Paths

Where controlled sharing is guaranteed (e.g., view types, spans, string_view), shallow copy avoids allocation overhead entirely.

When to Use Deep Copy

Deep Copy Is Required When:

- The class maintains **unique ownership** of a heap-allocated resource
- The resource is **mutable** — modifications in one copy must not affect another
- The resource represents an exclusive handle: a file descriptor, database connection, or socket
- Objects have independent, non-overlapping lifetimes

Real-World Scenarios

Dynamic arrays: A `Matrix` class managing its own `double*` buffer must deep copy to avoid two matrices sharing data.

File handles: Copying a class that wraps an open file should create a new, independently managed file reference.

String implementations: A custom string class must allocate and copy the character buffer — not just share the pointer.

When in doubt: if your destructor calls `delete`, you almost certainly need a deep copy constructor.

Implementing a Safe Copy Constructor

A correctly implemented copy constructor must allocate fresh resources, copy the data, and handle exceptions so that a partially constructed object does not leak memory. Below is a canonical example for a resource-owning class:

```
class Buffer {
  int* data;
  std::size_t size;
public:
  // Safe deep copy constructor
  Buffer(const Buffer& other)
    : size(other.size),
      data(new int[other.size]) // allocate first
  {
    std::copy(other.data,
              other.data + size,
              data);          // then copy data
  }
  ~Buffer() { delete[] data; }
};
```

- **Allocate before copying**

  Initialize `data` in the member initializer list so that if `new` throws, no partial state is left.

- **Use standard algorithms**

  `std::copy` is exception-safe and expressive — prefer it over raw loops.

- **Guard against self-copy**

  While less critical in the copy constructor, always be mindful: `if (this == &other) return;` in assignment operators.

Implementing a Move Constructor

Move constructors are the performance backbone of modern C++. The implementation follows a strict pattern: transfer, nullify, mark `noexcept`.

```
class Buffer {
  int* data;
  std::size_t size;
public:
  // Move constructor
  Buffer(Buffer&& other) noexcept
    : data(other.data),     // 1. Take ownership
      size(other.size)
  {
    other.data = nullptr;   // 2. Nullify source
    other.size = 0;         // 3. Leave source valid
  }
  ~Buffer() {
    delete[] data;          // Safe: nullptr delete is no-op
  }
};
```

Transfer

Copy the raw pointer from source to `this` — no allocation, O(1) cost.

Nullify

Set source pointer to `nullptr` so its destructor is harmless.

noexcept

Marking `noexcept` allows STL containers to use move over copy for optimal performance.

Example: Resource Class with Copy & Move

Here is a complete resource-managing class demonstrating both copy and move semantics side-by-side for clarity:

Copy Constructor (Deep)

```
class Resource {
  char* buf;
  int len;
public:
  Resource(const Resource& o)
    : len(o.len),
      buf(new char[o.len])
  {
    std::memcpy(buf, o.buf, len);
  }
};
```

Allocates a brand-new buffer and copies every byte. The two objects are completely independent.

Move Constructor (Transfer)

```
class Resource {
  char* buf;
  int len;
public:
  Resource(Resource&& o) noexcept
    : buf(o.buf), len(o.len)
  {
    o.buf = nullptr;
    o.len = 0;
  }
};
```

No allocation. Steals the buffer pointer in nanoseconds regardless of buffer size.

The Rule of Five in C++

If a class requires a user-defined **destructor**, it almost certainly requires explicit definitions for all five special member functions. This is the **Rule of Five** — a fundamental design principle for resource-managing classes in modern C++.

1

Destructor

Releases owned resources. The trigger that tells you the other four are needed.

2

Copy Constructor

Creates a new object as a deep copy of an existing one.

3

Copy Assignment Operator

Replaces an existing object's resources with a deep copy from another.

4

Move Constructor

Transfers resources from a temporary object into a new one.

5

Move Assignment Operator

Transfers resources from a temporary into an existing object, releasing its current resources first.

Why Follow the Rule of Five?

Prevents Resource Leaks

Without a proper destructor and copy semantics, resources can be leaked or freed twice. The Rule of Five closes every escape route for memory mismanagement.

Exception Safety

Properly implemented special members ensure that even when exceptions are thrown mid-operation, no resources are leaked and object invariants are maintained.

Optimized Performance

Defining move semantics allows the compiler and STL to use efficient transfers instead of costly copies — especially critical in container operations like `push_back`.

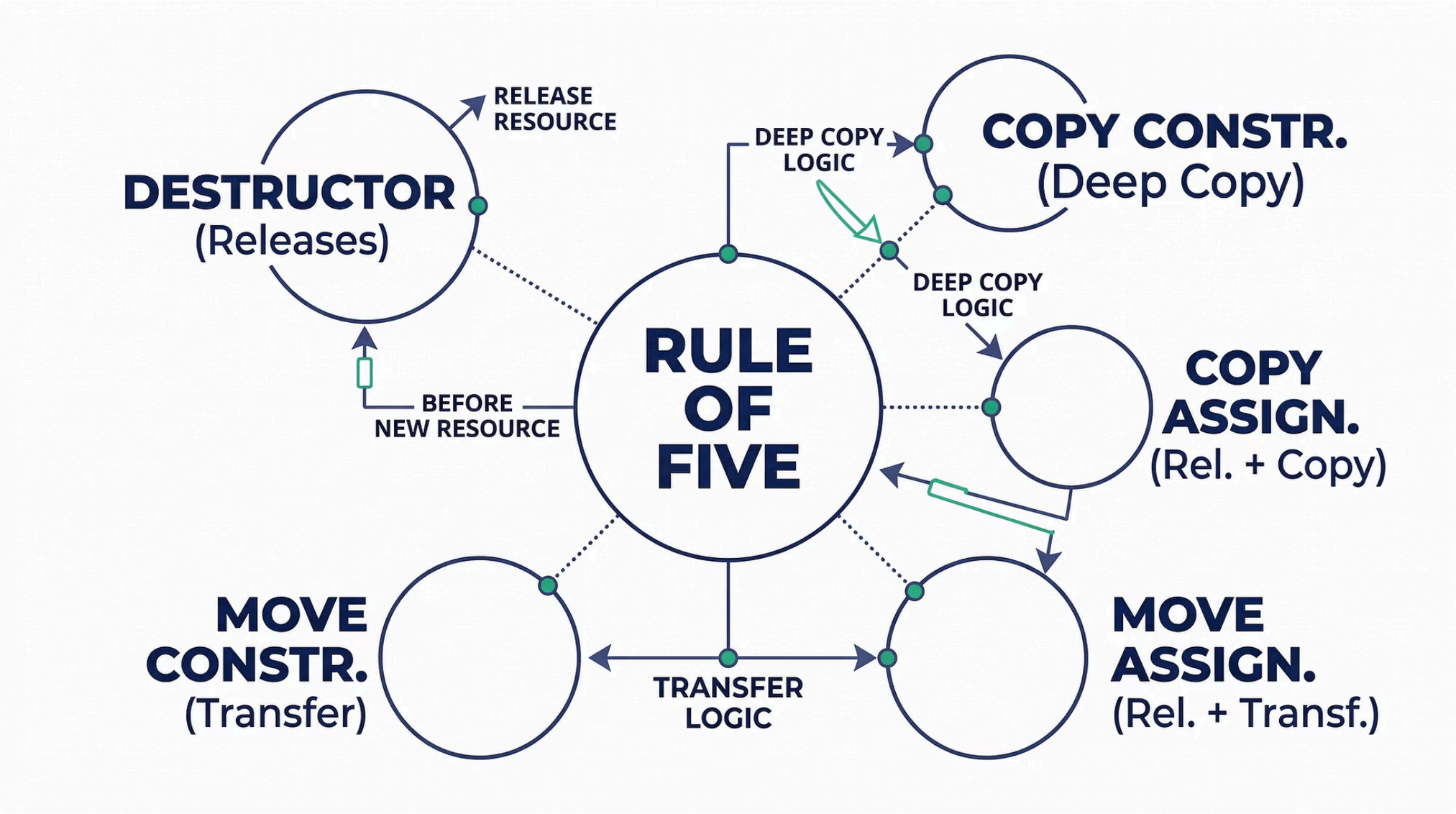
Implementing the Rule of Five

The canonical implementation uses RAII (Resource Acquisition Is Initialization) as its backbone. Every resource is tied to an object's lifetime. Below is the full Rule of Five skeleton:

```
class MyResource {
  int* data;
  std::size_t size;
public:
  // 1. Destructor
  ~MyResource() { delete[] data; }

  // 2. Copy Constructor
  MyResource(const MyResource& o);

  // 3. Copy Assignment
  MyResource& operator=(const MyResource& o);

  // 4. Move Constructor
  MyResource(MyResource&& o) noexcept;

  // 5. Move Assignment
  MyResource& operator=(MyResource&& o) noexcept;
};
```

Tip: If you use `std::unique_ptr` as the member, the compiler will correctly delete copy operations and generate move operations — effectively giving you the Rule of Five for free via the **Rule of Zero**.

Rule of Five: Member Interactions



The five special members are interdependent. The **destructor** is the central hub — assignment operators must release existing resources before acquiring new ones, making the destructor's logic reusable. Copy operations share deep-copy logic; move operations share transfer-and-nullify logic. Defining all five ensures every code path through an object's lifecycle is safe.

Common Pitfalls & Memory Safety Issues

Even experienced C++ developers fall into these traps when managing resources manually. Understanding them is the first step toward elimination.

🚫 Double Delete

Shallow copy causes two objects to share a pointer. When both are destroyed, `delete` is called twice on the same address — resulting in heap corruption or a crash.

🚫 Dangling Pointers After Move

If a moved-from object's pointer is not set to `nullptr`, its destructor will delete already-transferred memory. Always nullify source pointers in move operations.

🚫 Self-Assignment

In copy assignment (`a = a;`), releasing your own resources before copying from yourself deletes the data you're about to copy. Always check `if (this != &other)`.

🚫 Exception Leaks During Copy

If memory allocation in a copy constructor throws after partial construction, and no cleanup is performed, resources are leaked. Use member initializer lists and RAII to prevent this.

Diagnosing Memory Safety Risks

Dynamic Analysis Tools

- **Valgrind (Memcheck)**: Detects memory leaks, invalid reads/writes, and double-free errors at runtime with full stack traces.
- **AddressSanitizer (ASan)**: Compiler-instrumented runtime checker for buffer overflows, use-after-free, and stack issues. Enable with `-fsanitize=address`.
- **LeakSanitizer**: Focused specifically on memory leak detection, often bundled with ASan.

Static & Manual Analysis

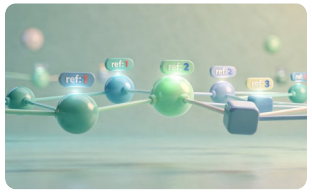
- **Clang-Tidy**: Applies C++ Core Guidelines checks, flagging classes that violate the Rule of Five.
- **Cppcheck**: Open-source static analyzer for common memory errors and style issues.
- **Code Reviews**: Systematic review focusing on ownership semantics — ask "who deletes this?" for every raw pointer.
- **Sanitizer flags**: Combine `-fsanitize=address,undefined` in CI pipelines for continuous coverage.

Improving Class Design for Safety

The most effective improvements to class safety involve replacing raw resource management with modern C++ abstractions that handle ownership automatically.

std::unique_ptr

Expresses **exclusive ownership**. Automatically deletes the resource when the pointer goes out of scope. Move-only by design — prevents accidental shallow copies at compile time.



std::shared_ptr

Expresses **shared ownership** via reference counting. Safe for multiple owners; resource is deleted only when the last owner is destroyed. Use sparingly — introduces overhead.

Move Semantics

Use `std::move()` explicitly to convert lvalues to rvalues when transferring ownership. Enables STL containers to relocate elements without copying, dramatically improving throughput.

The Rule of Zero

What is the Rule of Zero?

If a class uses members that already manage their own resources (smart pointers, STL containers, RAII wrappers), it should define **none** of the five special members. The compiler-generated defaults will be correct, safe, and efficient.

```
// Rule of Zero in practice
class Document {
  std::string title;
  std::vector<Page> pages;
  std::unique_ptr<Header> header;
  // No destructor needed!
  // No copy/move needed!
  // Compiler handles everything.
};
```

Benefits

- **Zero boilerplate**: No custom destructor, copy, or move code to write or maintain.
- **Correctness by construction**: Smart pointer members make illegal copies a compile-time error.
- **Fewer bugs**: Every line of manual resource management is a potential bug. Eliminating them eliminates entire bug classes.
- **Better readability**: Class definition expresses intent, not plumbing.

Rule of Thumb: Prefer Rule of Zero. Fall back to Rule of Five only when wrapping a truly raw resource (e.g., OS handle, C library).

Practical Example: Resource Wrapper Class

Here is a clean, modern resource wrapper that leverages `std::unique_ptr` to achieve full Rule of Zero compliance — with all the safety of the Rule of Five, automatically:

```
class SafeBuffer {
  std::unique_ptr<int[]> data;
  std::size_t size;

public:
  explicit SafeBuffer(std::size_t n)
    : data(std::make_unique<int[]>(n)), size(n) {}

  // Copy: explicitly deep copy
  SafeBuffer(const SafeBuffer& o)
    : data(std::make_unique<int[]>(o.size)), size(o.size) {
    std::copy(o.data.get(), o.data.get() + size, data.get());
  }

  // Move: defaulted — unique_ptr handles it
  SafeBuffer(SafeBuffer&&) noexcept = default;
  SafeBuffer& operator=(SafeBuffer&&) noexcept = default;

  // Copy assignment
  SafeBuffer& operator=(const SafeBuffer& o);

  // Destructor: unique_ptr deletes automatically
  ~SafeBuffer() = default;
};
```

`std::unique_ptr` handles destruction automatically. No manual `delete`, no leaks, no double-frees.

Best Practices Summary

01

Define All Five When You Define Any

If you write a destructor, copy constructor, or assignment operator, define all five special members explicitly. Never leave any to chance.

02

Use RAII and Smart Pointers

Tie resource lifetimes to object lifetimes using `std::unique_ptr`, `std::shared_ptr`, or standard containers. Eliminate raw `new`/`delete` from application code.

03

Prefer the Rule of Zero

Design classes so the compiler can generate correct special members. Reserve manual implementation for low-level resource wrappers only.

04

Mark Move Operations noexcept

This enables STL optimizations. A move constructor that can throw forces containers like `std::vector` to fall back to copying during reallocation.

05

Use Tools Continuously

Integrate AddressSanitizer and static analyzers into your CI pipeline. Catching leaks and invalid accesses early is far cheaper than debugging in production.

Case Study: Evaluating a Class Design

Let's analyze a typical **unsafe** class design using raw pointers and identify every issue:

```
//  UNSAFE: Violates Rule of Five
class DataStore {
  int* buffer;
  int  count;
public:
  DataStore(int n)
    : buffer(new int[n]), count(n) {}

  ~DataStore() { delete[] buffer; }

  // No copy constructor  → shallow copy by default
  // No copy assignment   → shallow copy by default
  // No move constructor  → copy used instead (slow)
  // No move assignment   → copy used instead (slow)
};
```

Issue 1

Default copy constructor copies the raw pointer — two objects share one buffer → double-free on destruction.

Issue 2

No copy assignment → self-assignment deletes buffer before copying, causing use-after-free.

Issue 3

No move semantics → every transfer is a slow deep copy or a dangerous shallow one.

Before & After: Class Design Improvements

❌ Before: Raw Pointer, Unsafe

```
class DataStore {
  int* buffer;
  int  count;
public:
  DataStore(int n)
    : buffer(new int[n]),
      count(n) {}
  ~DataStore() {
    delete[] buffer;
  }
  // Missing: copy ctor,
  // copy assign,
  // move ctor, move assign
  // → Undefined behavior!
};
```

Shallow copy by default. Double-free waiting to happen. No move optimization.

✅ After: Smart Pointer, Safe

```
class DataStore {
  std::unique_ptr<int[]> buffer;
  int count;
public:
  DataStore(int n)
    : buffer(
        std::make_unique<int[]>(n)
      ), count(n) {}

  // Copy: deep copy
  DataStore(const DataStore& o);
  DataStore& operator=(
    const DataStore& o);

  // Move: default is correct
  DataStore(DataStore&&) = default;
  DataStore& operator=(
    DataStore&&) = default;
  // Destructor: automatic!
};
```

No leaks. No double-free. Move is O(1). Copy is safe and explicit.

Summary & Key Takeaways

Copy vs. Move

Copy constructors create independent duplicates of an object's resources. **Move constructors** transfer ownership from an expiring object — O(1) cost, no allocation.

Shallow vs. Deep

Shallow copy is only safe for shared, immutable, or externally managed resources. **Deep copy** is required for unique ownership — always allocate new memory for owned pointers.

Rule of Five

If you define *any* of the five special members (destructor, copy ctor, copy assign, move ctor, move assign), define *all* of them. No exceptions.

Modern C++ Approach

Use `std::unique_ptr`, `std::shared_ptr`, and STL containers to apply the Rule of Zero. Let the type system enforce ownership — not runtime luck.

Final Thoughts

> "Resource management is not a detail — it is the foundation upon which all correct, efficient C++ programs are built."

Proper copy and move semantics are not optional features for "advanced" code — they are fundamental responsibilities of any class that owns a resource. The tools are all there: smart pointers, RAII, move semantics, and modern compiler support. The only remaining ingredient is **discipline**: applying these patterns consistently, using analysis tools, and designing with ownership in mind from the first line of code.

Embrace modern C++17/20 features. Write classes that are correct by construction. Your future self — and your team — will thank you.

Resources & Further Reading

Stack Overflow

C++ Rule of 5 — Copy and Move
stackoverflow.com/questions/29465527

cppreference.com

Rule of Three / Five / Zero — the definitive reference for C++ special member behavior and compiler rules.
cppreference.com/w/cpp/language/rule_of_three

C++ Core Guidelines

Bjarne Stroustrup & Herb Sutter's living document of modern C++ best practices, including extensive coverage of resource management, ownership, and RAII.

Effective Modern C++ — Scott Meyers

42 specific ways to improve your use of C++11 and C++14, including deep dives into move semantics, perfect forwarding, and smart pointers.

Call to Action

Don't let another class ship without asking the right ownership questions. Here are three concrete actions you can take **today**:

1

Audit Your Classes

Find every class in your codebase that has a destructor. Check whether it defines all five special members. Flag any that don't for immediate review.

2

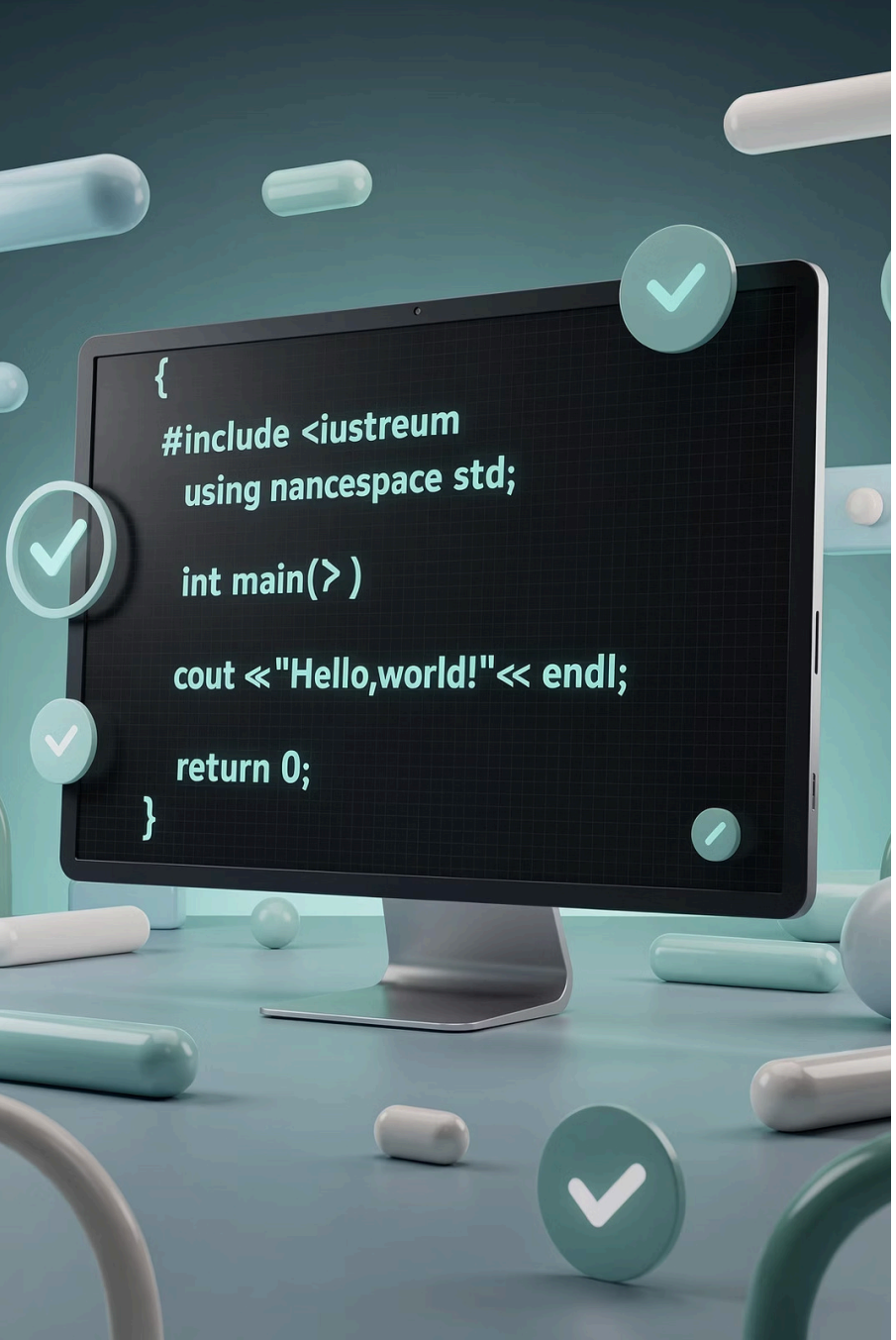
Adopt Smart Pointers

Replace raw `new`/`delete` with `std::unique_ptr` or `std::shared_ptr`. Apply the Rule of Zero wherever possible to eliminate boilerplate.

3

Enable Sanitizers in CI

Add `-fsanitize=address,undefined` to your build pipeline. Run Valgrind or Clang-Tidy on every pull request. Make memory safety a gating requirement.



Thank You!

You now have the knowledge to design C++ classes that are **safe, efficient, and correct** by construction. From copy constructors to the Rule of Five, these patterns are the difference between fragile code and production-grade software.

🤔 Questions?

Let's discuss any of the patterns, code examples, or design decisions covered today.

💻 Happy Coding!

Write safe, move-semantic-powered, RAII-driven C++ that your team will be proud of.

RULE OF FIVE | RAII | SMART POINTERS | MOVE SEMANTICS